

Workshop: IA pra Desenvolvimento de Software

Da base à prática com agentes

Carga horária: ~16 horas (2 dias)

Pra quem: Desenvolvedores com experiência profissional

Formato: Teoria + laboratórios práticos (30-50 min por módulo)

Referências e glossário: No final do documento

Pré-requisitos

Ferramenta obrigatória:

- [OpenCode](#) instalado — use a assinatura **FREE** (gratuita) ou o **Plano Básico de US\$ 5 no primeiro mês** para ter acesso a mais recursos. O OpenCode será usado como ferramenta principal durante todos os laboratórios do workshop. **Ferramentas recomendadas (instalamos juntos durante o workshop):**
- **Git e Docker** — para laboratórios de MCP e SDD
- **Conta no GitHub** — para laboratórios de integração com MCP
- **Uma IDE com suporte a IA** — VS Code + Terminal com Opencode, GitHub Copilot, ou Claude Code, ou outros. **Nivelamento recomendado (não obrigatório):**
- Assistir os 3 vídeos de nivelamento (~1h30 total): 1. Roadmap de IA para DevOps (20 min) 2. Prompt Engineering — Guia Prático (1h10) 3. SDD: Habilidade #1 para Devs (12 min)

O objetivo

Dar a devs o que precisam pra usar IA generativa com segurança e produtividade no ciclo completo de desenvolvimento — entendendo os fundamentos técnicos, dominando as técnicas de interação e orquestrando agentes em projetos reais.

Conteúdo Programático

Encontro	Módulo	Tema	Duração	Laboratório
1	M1 — Como LLMs Funcionam de Verdade	Autocomplete, tokens, limitações, fine-tuning vs RAG	2h	30 min — Diagnosticar e mitigar limitações
1	M2 — Ecossistema Open-Source e Execução Local	Ollama, Hugging Face, parâmetros, nuvem vs local	1h30	30 min — Rodar modelo local vs API na nuvem
2	M3 — Prompt Engineering na Prática	5 elementos, RTF/CARE/RISE, Zero/Few-shot, CoT	2h30	45 min — Oficina de prompts em 3 cenários reais
2	M4 — RAG, Embeddings e Engenharia de Contexto	Pipeline RAG, embeddings, Graph RAG, progressive disclosure	2h	30 min — Visualizar embeddings e pipeline RAG
3	M5 — MCP e Agentes de IA	Arquitetura MCP, agentes, loop de execução, segurança de MCP, MCP vs Skill vs API	1h46	Demos guiadas (23 min) — MCP do zero + Playwright · lab no kit pós-seminário
3	M6 — Arquitetura de Contexto	Rules/Agents.md, Skills modulares, sub-agents, checklist dos 4 pilares	1h49	Demo guiada (12 min) — onboarding num legado real · lab no kit pós-seminário

Encontro	Módulo	Tema	Duração	Laboratório
4	M7 — SDD e Ferramentas na Prática	4 fases do SDD, STATE.md, frameworks (OpenSpec/TLC/Kiro), RPI, validação de código	4h	Construção ao vivo (1h55) — Specify→Tasks · Execute real · Validar · Quando dá errado

Totais: 16h — 4 encontros de 4h (sextas-feiras), online · 7 módulos

Encontros 1 e 2 — Fundamentos e Técnicas (8h)

Módulo 1 — Como LLMs Funcionam de Verdade (2h)

O que você vai aprender:

- Explicar LLMs usando o modelo mental do autocomplete
- Calcular tokens em cenários reais
- Identificar e mitigar os 4 tipos de limitação
- Decidir entre modelo pronto, fine-tuning ou RAG
- Saber quando uma LLM é a ferramenta certa — e quando não é **Tópicos:**

1.1 O Grande Autocomplete

A ideia: Uma LLM não "pensa" — ela completa padrões. Pensa no autocomplete do celular: você digita "bom..." e ele sugere "dia", "tarde", "noite" baseado no que já escreveu. A LLM faz a mesma coisa, mas em vez de 3 opções, escolhe entre milhões, calculando o token mais provável dado o que veio antes. Não tem intenção, opinião, nem "saber" coisas — é uma máquina de previsão estatística de tokens.

Por que isso importa: Muda como você escreve prompts e interpreta respostas. Você para de tratar a IA como oráculo e passa a tratar como um completador de padrões que precisa de contexto de qualidade.

Pra visualizar: Autocomplete turbinado — milhões de opções em vez de 3, e ele lembra de tudo na conversa.

1.2 Modelos Multimodais

A ideia: Modelos multimodais entendem texto, imagem, áudio e vídeo. Dá pra enviar um print de bug, um diagrama rabiscado ou um áudio com sua dúvida, e o modelo processa tudo junto. GPT-4o, Gemini e Claude fazem isso.

Por que isso importa: Expande o que você pode delegar — explicar um erro a partir de um screenshot, gerar código de um wireframe, transcrever e analisar uma reunião.

Pra visualizar: Um dev que recebia specs só por texto agora recebe fotos do problema, áudios do cliente e vídeos do comportamento. Exemplo aplicação web rodando no Google Chrome com gemini-nano multimodal.

1.3 Tokens: A Moeda da IA

A ideia: Token é a unidade de processamento, cada palavra ou fragmento que o modelo lê ou gera. "Inteligência artificial" pode ser 2 tokens. Toda API cobra por token (input + output). Uma conversa típica de desenvolvimento consome entre 2 mil e 20 mil tokens. Modelos maiores têm janelas maiores (Gemini 1.5 Pro: 2M tokens $\approx$ 3 livros). Mas tem uma pegadinha: se você lotar a janela, o modelo começa a "esquecer" coisas no meio.

Por que isso importa: Token = custo. Quem entende tokenização consegue estimar gastos, otimizar prompts e evitar desperdício com contexto redundante. Também ajuda a não estourar a janela e degradar a qualidade.

Pra visualizar: Token é que nem kilowatt, a moeda de consumo. A janela de contexto é a RAM: você pode encher, mas o desempenho piora antes do limite.

Prompt Caching: As principais APIs (Anthropic, OpenAI, Google) cacheiam automaticamente instruções repetidas e cobram $\sim 90\%$ menos. É tipo Git: a primeira vez que você clona um repositório, paga o custo cheio. Depois, `git pull` só baixa o que mudou, quase de graça. A técnica #1 pra reduzir custos com IA em 2025-2026, e a conexão direta entre custo (aqui no M1) e arquitetura de contexto (Módulo 6).

1.4 As 4 Grandes Limitações

A ideia:

1. **Alucinações:** a IA inventa resposta com toda confiança, mas ela é falsa. Tipo pedir artigos de um autor e ela criar títulos que não existem.
2. **Data de corte:** a IA é uma enciclopédia impressa em 2023 — não sabe nada depois, a menos que você dê acesso à internet ou alimente com documentos recentes.
3. **Sensibilidade ao prompt:** mudar uma palavra na pergunta pode gerar resposta completamente diferente. "Explique RAG" vs "Explique RAG pra uma criança de 10 anos" — resultados totalmente diferentes.
4. **Degradação com contexto cheio:** quanto mais informação você joga de uma vez, mais a IA se perde no meio. **Por que isso importa:** Conhecer essas limitações separa quem culpa a IA ("não presta") de quem contorna com estratégia ("vou quebrar em partes", "vou validar", "vou usar RAG").

Pra visualizar: A IA é tipo um estagiário brilhante mas com amnésia seletiva. Inventar coisas com convicção (alucina), não sabe o que aconteceu depois que foi contratado (corte), responde diferente dependendo de como você pergunta (sensibilidade) e se perde com instruções demais (sobrecarga).

1.5 Pré-Treinamento, Fine-Tuning

A ideia: GPT, Claude e Gemini são formados — fizeram graduação. Eles sabem fazer de tudo de forma genérica com conteúdo que aprenderam, mas não sabem como o SIG funciona por exemplo.

Fine-tuning é quando você pega um modelo open-source (tipo Llama 8B) e mostra pra ele centenas de exemplos do padrão que você quer. Exemplo concreto: você tem 500 repositórios Java do seu banco. Todos seguem a mesma convenção — classes `XxxService`, DTOs `XxxDTO`, testes com `@DisplayName`, exceptions wrappadas em `BusinessException`. Você alimenta o modelo com centenas de pares "entrada → saída esperada" nesse padrão. Depois de ver isso 500 vezes, ele aprende. Aí quando o dev digita "cria o endpoint de estorno", o modelo gera tudo no padrão do banco — sem precisar explicar a convenção a cada prompt.

O problema? Requer GPU, dados rotulados de qualidade e retreino sempre que o padrão muda. Pra 95% dos casos, é canhão demais pra uma formiga.

RAG resolve a mesma coisa sem treinar nada: em vez de fazer o modelo decorar seu conhecimento específico, você entrega padrões pra ele na hora que for usar. É

mais barato, mais rápido e atualiza instantaneamente.

Por que isso importa: Saber que fine-tuning existe é importante — mas saber que RAG resolve a maioria dos casos sem GPU, sem retreino e sem dor de cabeça é mais importante ainda. Antes de cogitar fine-tuning: "Eu já tentei RAG?"

Pra visualizar: Pré-treinamento = profissional formado. Fine-tuning = você pagar pra ele decorar seu sistema inteiro. RAG = entregar a doc pra ele na hora que precisar.

1.6 Quando NÃO Usar uma LLM

A ideia: Nem tudo precisa de IA. Um algoritmo determinístico resolve melhor e mais barato em muitos cenários: validação de formato, parsing, cálculos, ordenação, filtros simples. Saber quando NÃO usar LLM é tão importante quanto saber usar.

Por que isso importa: Evita o viés do martelo dourado — usar LLM pra tudo é caro, lento e introduz não-determinismo onde você não precisa.

Pra visualizar: Você não chama um arquiteto pra pendurar um quadro. Às vezes um prego e um martelo resolvem.

Laboratório (30 min): Cada participante recebe 3 prompts com problemas diferentes — alucinação, data de corte, sobrecarga de contexto. A tarefa: diagnosticar qual limitação está rolando em cada caso e reescrever o prompt pra mitigar. Bônus: calcular os tokens de cada versão e comparar o custo.

Módulo 2 — Ecossistema Open-Source e Execução Local (1h30)

O que você vai aprender:

- Conhecer a possibilidade de rodar um modelo open-source localmente via Ollama
- Navegar no Hugging Face pra encontrar modelos adequados
- Comparar modelos por custo, latência, privacidade e capacidade
- Decidir entre execução local vs API na nuvem

Tópicos:

2.1 Modelos Open-Source: As Receitas Públicas

A ideia: Modelos open-source (Llama da Meta, Qwen da Alibaba, DeepSeek, Mistral) são receitas de bolo publicadas de graça — você pode usar, modificar e rodar na sua cozinha sem pagar royalties. Já os proprietários (GPT, Claude, Gemini) são restaurantes: a comida é ótima, mas você não tem acesso à receita nem controle sobre a cozinha.

Por que isso importa: Open-source é soberania — você não depende de fornecedor, seus dados não trafegam por APIs de terceiros e pode customizar o modelo. Pra empresas com restrições de compliance, é muitas vezes a única opção.

Pra visualizar: Proprietário = restaurante. Open-source = receita que você cozinha em casa.

2.2 Ollama: O Eletrodoméstico

A ideia: Ollama faz execução local de LLMs ser trivial — parecido com Docker pra modelos de IA. Instala, escolhe o modelo (`ollama run llama3.2`), e roda na sua máquina. Sem depender de internet, sem pagar API, sem enviar dados pra lugar nenhum.

Por que isso importa: Elimina a barreira de entrada pra testar modelos open-source. Em 5 minutos você roda um modelo localmente, compara com APIs e decide com dados reais.

Pra visualizar: Ollama tá pra LLM assim como Docker tá pra aplicação — baixa e roda com um comando.

2.3 Hugging Face: O GitHub dos Modelos

A ideia: Hugging Face é o maior repositório de modelos de IA — milhares de modelos pré-treinados, datasets e versões modificadas pela comunidade. É onde você encontra modelos especializados (um treinado pra SQL, outro pra revisão de contratos) e benchmarks.

Por que isso importa: Antes de pensar em treinar qualquer coisa, procure no Hugging Face. Provavelmente alguém já fez algo similar. Economiza semanas.

Pra visualizar: Hugging Face tá pra modelo de IA assim como GitHub tá pra código — repositório público, comunidade ativa, versões, forks.

2.4 Parâmetros: + Nem Sempre é Melhor

A ideia: Parâmetros são tipo os "neurônios" da rede — quanto mais, maior a capacidade teórica. Mas a correlação não é linear. Pesquisas recentes mostram que remover parâmetros às vezes melhora o modelo (tipo tirar "sujeira" do cérebro). Modelos menores e especializados (Phi-3, Llama 8B) entregam 80% da qualidade de um modelo grande por 10% do custo.

Por que isso importa: Nem toda tarefa precisa de um modelo caro. Pra sumarização, classificação, parsing e código simples, um SLM local resolve — com latência menor e custo zero de API.

Pra visualizar: Você não precisa de um caminhão pra ir na padaria. SLM = moto. LLM gigante = caminhão.

2.5 Critérios de Escolha: Nuvem vs Local

A ideia: A decisão entre API na nuvem e execução local depende de 4 coisas: **custo** (API cobra por token, local sai de graça depois do setup), **latência** (local é mais rápido pra tarefas simples), **privacidade** (local = dados não saem da máquina) e **capacidade** (nuvem tem modelos maiores pra raciocínio complexo).

Por que isso importa: Times que entendem esse trade-off montam arquiteturas híbridas — SLM local pra tarefas de alto volume e baixo risco (autocomplete, linting), API na nuvem pra tarefas complexas e pontuais (design de arquitetura, debugging multi-arquivo).

Laboratório (30 min): Instalar Ollama, baixar um modelo pequeno (Llama 3.2 3B ou Qwen 2.5 7B), executar 3 prompts idênticos no modelo local e em uma API na nuvem (Claude ou GPT). Comparar qualidade, latência e custo estimado. Preencher uma matriz de decisão.

Módulo 3 — Prompt Engineering na Prática (2h30)

O que você vai aprender:

- Construir prompts com os 5 elementos (Role, Contexto, Instrução, Restrições, Formato)
- Escolher o framework certo (RTF, CARE, RISE) pra cada tarefa
- Aplicar Zero-shot, Few-shot e Chain of Thought no contexto certo
- Depurar e iterar prompts ineficazes **Tópicos:**

3.1 Os 5 Elementos de um Prompt Eficiente

A ideia: Um prompt bem construído tem 5 camadas:

1. **Role:** "Você é um dev sênior especialista em refatoração de código legado" — direciona tom e nível técnico.
2. **Contexto:** "Este projeto é uma API Go 1.22 com arquitetura hexagonal" — o background que a IA não tem.
3. **Instrução:** "Refatore a função extraíndo a validação pra outra função" — a ação, clara e direta.
4. **Restrições:** "Mantenha os nomes de função. Não use generics." — limites e o que não fazer.
5. **Formato de saída:** "Retorne só o código em markdown com o nome do arquivo." — estrutura esperada. **Por que isso importa:** Um prompt com os 5 elementos produz respostas melhores que um prompt na base do achismo. A diferença não é pequena — é entre receber código que você pode commitar e código que você reescreve do zero.

Pra visualizar: Pedir sem estrutura = "conserta minha casa". Com os 5 elementos = entregar planta baixa, materiais, normas e prazo.

3.2 Frameworks de Prompt: RTF, CARE, RISE

A ideia: Nem todo prompt precisa dos 5 elementos. Dá pra usar frameworks mais enxutos:

- **RTF (Role, Task, Format):** pra tarefas simples e de padrão conhecido. Gerar Dockerfile, JSON schema, script SQL.
- **CARE (Context, Action, Result, Example):** pra tarefas com objetivo de negócio. Runbook de incidente, plano de migração.
- **RISE (Role, Input, Steps, Example):** pra tarefas com múltiplos passos (Chain of Thought implícito). Análise de vulnerabilidade, troubleshooting. **Por que isso importa:** Economiza tokens e tempo. Você não precisa dos 5 elementos pra gerar um Dockerfile — RTF resolve com 3 linhas.

Pra visualizar: RTF = post-it. CARE = e-mail formal. RISE = documentação técnica passo a passo.

3.3 Técnicas de Raciocínio

A ideia: Os frameworks estruturam o prompt; as técnicas definem como o modelo raciocina:

- **Zero-shot:** instrução direta, sem exemplo. Pra tarefas comuns que o modelo já conhece (CRUD, explicar erro, traduzir código).
- **Few-shot:** você dá 1-3 exemplos de input → output. Pra formato muito específico ou convenção interna que o modelo não conhece.
- **Chain of Thought:** força o modelo a mostrar o raciocínio passo a passo antes da resposta final. Pra análise, troubleshooting, decisões com trade-off. **Por que isso importa:** Few-shot é a técnica mais subestimada — 2 exemplos bem escolhidos entregam mais qualidade que 2 horas refinando prompt. Chain of Thought é a que mais reduz alucinação em tarefas analíticas.

Pra visualizar: Zero-shot = "faz um bolo". Few-shot = "faz um bolo igual a este" (mostra foto). Chain of Thought = "me explica como você vai fazer antes de começar".

3.4 Antipadrões e Erros Comuns

A ideia: Os erros mais frequentes:

- **Sobrecarga de instruções:** 10 comandos num prompt só — o modelo prioriza os primeiros e ignora os últimos.
- **Prompt vago demais:** "melhore esse código" — o modelo não sabe se é performance, legibilidade ou segurança.
- **Restrições contraditórias:** "seja conciso" + "explique em detalhes" — o modelo trava.
- **Confiar cegamente:** não validar saídas, especialmente código e dados estruturados. **Por que isso importa:** Conhecer os antipadrões reduz o tempo de iteração. A maioria dos prompts ruins sofre de um desses 4 problemas.

Laboratório (45 min): Pipeline de análise de feedbacks de usuários de um app financeiro em dois passos encadeados:

1. **Sanitização:** usar o prompt `data-sanitizer.md` com a base `base_reclamacoes.json` para gerar um dataset limpo, anonimizado e sem ruído (registros de bot, testes, reclamações fora do escopo do produto).
2. **Classificação e backlog:** usar o JSON gerado no Passo 1 como entrada no `insights-distiller.md` para extrair um backlog priorizado — bugs críticos,

melhorias de UX e novas features — com severidade e ação proposta para cada item.

Resultado salvo em `resultado_pipeline_feedback_analyser.md` e `backlog.json`.

Discussão coletiva: o que o modelo descartou, o que classificou de forma inesperada e se as ações propostas seriam acionáveis para um time de engenharia.

Módulo 4 — RAG, Embeddings e Engenharia de Contexto (2h)

O que você vai aprender:

- Explicar cada etapa do pipeline RAG e seu propósito
- Entender embeddings como representação vetorial de similaridade semântica
- Diferenciar RAG tradicional de Graph RAG e escolher a abordagem certa
- Aplicar o princípio de progressive disclosure **Tópicos:**

4.1 RAG: A Cola na Prova

A ideia: RAG (Retrieval-Augmented Generation) resolve a maioria dos casos em que a IA precisa de conhecimento específico do seu domínio — sem retreinar nada. Em vez de confiar na memória da IA (que alucina, falha, desatualiza), você entrega os trechos relevantes do seu material junto com a pergunta. Tipo dar uma "cola" pra IA durante a prova.

Por que isso importa: RAG é mais barato, mais rápido e mais fácil de atualizar que fine-tuning. Mudou a documentação? Reindexa. Não precisa retreinar nada. Para usos corporativos (FAQ, docs internas, base de conhecimento), RAG resolve.

Pra visualizar: Fine-tuning = mandar o aluno estudar o livro inteiro antes da prova. RAG = deixar o livro aberto na página certa.

4.2 O Pipeline RAG em 5 Etapas

A ideia: O pipeline RAG completo:

1. **Chunks:** quebrar documentos grandes em pedaços menores e coerentes. Um manual de 500 páginas vira centenas de blocos de 500-1000 tokens.
2. **Embeddings:** transformar cada chunk num vetor numérico — uma lista de números que representa a posição daquele texto no espaço semântico.

3. **Vector Database:** armazenar esses vetores num banco otimizado pra busca por proximidade (Chroma, Pinecone, pgvector).
4. **Busca por similaridade:** quando o usuário pergunta algo, a pergunta vira embedding também. O banco encontra os 3-5 chunks mais próximos (cosseno)
5. **Injeção no prompt:** esses chunks viram contexto no prompt junto com a pergunta. O modelo responde baseado nesse contexto, não na memória. **Por que isso importa:** Entender o pipeline permite debugar quando o RAG falha. Resposta ruim? Pode ser chunk mal dimensionado, embedding de baixa qualidade, threshold errado ou chunk irrelevante sendo injetado.

Pra visualizar: É uma biblioteca. Chunks = fichas. Embeddings = coordenadas da estante. Vector DB = sistema de busca. Similaridade = encontrar livros na mesma estante. Injeção = colocar os livros abertos na mesa do aluno.

4.3 Embeddings: Como a IA "Entende" Palavras

A ideia: Embeddings transformam palavras em listas de números — tipo coordenadas GPS no espaço do significado. "Cachorro" e "gato" ficam perto (são pets). "Cachorro" e "carro" ficam longe. Num mapa 2D: "rei" e "rainha" estariam próximos; "homem" e "mulher" também, mas em outra região.

Sem embeddings, a IA trataria "automóvel" e "carro" como palavras diferentes.

Por que isso importa: Embeddings são a base invisível de quase toda aplicação prática de IA — busca semântica em código, recomendação de docs.

Pra visualizar: Exemplo de busca com CEP para identificar qual estado ele pertence em uma base de dados.

4.4 Graph RAG: Quando Similaridade Não Basta

A ideia: RAG tradicional busca por similaridade de texto — funciona pra perguntas factuais ("quanto custa o plano X?"). Mas falha em perguntas que exigem conexões entre entidades ("quantos clientes do plano X também usam o produto Y?"). Graph RAG resolve montando um grafo de conhecimento (entidades como nós, relacionamentos como arestas) e navegando por ele com consultas tipo Cypher (Neo4j).

Por que isso importa: Graph RAG é a abordagem certa pra domínios com muitas conexões: catálogos de produtos com dependências, APIs com compatibilidade, bases de conhecimento com cross-references.

Pra visualizar: RAG tradicional = buscar "restaurante italiano" no Google. Graph RAG = "restaurantes italianos a menos de 1 km, abertos agora, nota acima de 4.5" — cada condição é uma aresta no grafo.

4.5 Engenharia de Contexto e a Regra dos 50/70%

A ideia: Contexto é o maior diferencial no uso de IA — mais que o modelo escolhido. Mas tem um limite: a **regra dos 50-70%** diz pra usar no máximo 70% da janela de contexto. Se o modelo suporta 200 mil tokens, mantenha abaixo de 140 mil. Acima disso, o modelo degrada.

Progressive disclosure resolve isso: carrega só o necessário, só quando necessário — como um garçom que traz primeiro a entrada, depois o prato principal, depois a sobremesa, não o pedido inteiro.

Por que isso importa: É a diferença entre uma IA que acerta e uma que alucina, com os mesmos modelo e pergunta. A qualidade do contexto impacta mais que o modelo ou o prompt.

Pra visualizar: Sua mesa de trabalho. 50 documentos = caos. 3 documentos = foco total.

Laboratório (30 min):

1. Visualizar embeddings: identificar proximidade e estado pelo CEP.
2. Explicar pipeline RAG simplificado: quebrar documento, gerar embeddings, buscar trecho similar a uma pergunta, injetar no prompt e comparar resposta com vs sem RAG.

Encontros 3 e 4 — Ferramentas de Contexto e Prática (8h)

Módulo 5 — MCP e Agentes de IA (2h)

O que você vai aprender:

- Explicar a arquitetura MCP Server/Client e seu papel como padrão de integração
 - Identificar os 4 componentes de um agente de IA
 - Descrever o loop de execução de um agente e como ele difere de um chat
 - Configurar um MCP Server e ver o agente descobrindo ferramentas
 - Aplicar as práticas mínimas de segurança ao conectar um agente a sistemas corporativos
- Tópicos:**

5.1 MCP: A Tomada Universal

A ideia: MCP (Model Context Protocol, da Anthropic) resolveu um problema real: antes, conectar uma IA no banco de dados, Jira e GitHub era um pesadelo — cada combinação de ferramenta + modelo exigia código customizado. MCP é tipo tomada universal: qualquer ferramenta que segue o padrão conecta em qualquer IA que também segue. Hoje é padrão da indústria — Cursor, Claude Code, Copilot.

Por que isso importa: Elimina o trabalho de integração. Configura um MCP Server pro Jira uma vez, e toda IA compatível descobre e usa. O protocolo gerencia autenticação e descoberta de capacidades.

Pra visualizar: Antes do MCP = cada eletrodoméstico com sua tomada. Depois = tomada universal de 3 pinos.

5.2 Arquitetura Server/Client

A ideia: O ecossistema MCP tem dois lados:

- **MCP Server:** expõe capacidades — "sei buscar e-mails", "sei criar tasks no Jira", "sei consultar logs". É um adaptador entre uma API externa e o protocolo.
- **MCP Client:** descobre as capacidades dos servidores e gerencia as chamadas. É o intermediário entre o modelo e os servidores. Exemplos: Claude Code, Cursor, Copilot. O fluxo: a IA decide "preciso buscar a task CAN-123" → o Client vê que o servidor do Jira expõe `getTask` → o servidor traduz pra API do Jira → o resultado volta.

Por que isso importa: Entender a arquitetura permite depurar problemas de integração e decidir quando usar MCP vs API direta.

Pra visualizar: MCP Server = garçom. MCP Client = maître que sabe quem tá disponível. IA = cliente.

5.3 Agentes automatizados: Além do Chat

A ideia: A diferença entre chat e agente: no chat você pergunta "qual a previsão do tempo?", a IA responde. Um agente recebe "me avisa se chover amanhã e remarca minhas reuniões externas" — ele planeja, executa (consulta previsão → verifica agenda → identifica reuniões → remarca → notifica) e verifica se atingiu o resultado.

Um agente tem 4 componentes:

1. **Cérebro:** a LLM que raciocina e decide.
2. **Mãos:** ferramentas (MCPs, APIs) que executam ações.
3. **Memória:** contexto da conversa + banco externo pra persistência.
4. **Guardrails:** regras de segurança — "nunca delete sem confirmar". **Por que isso importa:** Agentes são o próximo salto de produtividade. Um chat responde; um agente executa. Em IDEs modernas, o modo agente significa que a IA lê código, planeja mudanças, edita arquivos, roda testes e itera — tudo de uma vez.

Pra visualizar: Chat = perguntar as horas. Agente = assistente que olha sua agenda, calcula o trânsito, te avisa e já adianta o café.

5.4 O Loop do Agente

A ideia: O ciclo de execução de um agente:

1. **Planejar:** a LLM analisa o objetivo e define os passos.
2. **Executar:** chama as ferramentas (MCPs) pra cada passo.
3. **Observar:** coleta e analisa os resultados.
4. **Replanejar:** se algo falhou, ajusta o plano e continua. Esse loop roda até o objetivo ser atingido ou um guardrail interromper.

Por que isso importa: Entender o loop permite criar agentes mais eficazes — você antecipa onde ele pode travar e adiciona fallbacks.

Pra visualizar: É o PDCA da qualidade, executado por IA em segundos em vez de humanos em semanas.

5.5 Segurança de MCP

A ideia: Conectar um agente a sistemas corporativos abre superfície nova. Quatro práticas cobrem a maior parte do risco:

1. **Usuário read-only para MCP de banco.** Não é recomendação, é regra — o agente não precisa de `DELETE` pra responder uma pergunta.
2. **Token em variável de ambiente, nunca no `.mcp.json` versionado.** O arquivo de config vai pro git; o segredo não.
3. **Allowlist e modo de permissão.** Ferramenta de leitura libera; ferramenta de escrita pede confirmação; ferramenta destrutiva nem entra na config.
4. **Prompt injection via resultado de ferramenta.** O que volta do MCP é *dado*, mas o modelo lê como *texto*. Se um usuário externo escrever instruções dentro de um chamado, comentário ou formulário, o agente pode obedecer.

Por que isso importa: O item 4 é o menos conhecido e o mais perigoso. Qualquer sistema em que usuário externo escreve texto livre — chamado, formulário, comentário — virou superfície de ataque no momento em que um agente passou a ler aquele texto.

Pra visualizar: O MCP é uma porta. Segurança de MCP é decidir quem tem a chave, o que dá pra levar pra fora e o que não pode entrar junto com a encomenda.

5.6 MCP vs Skill vs API Direta

A ideia: Qual usar pra cada tipo de integração:

- **MCP:** dados que MUDAM (status de tasks, páginas do Confluence, PRs). O protocolo gerencia autenticação. Ideal pra sistemas corporativos.
- **Skill:** conhecimento ESTÁVEL (como criar teste e2e, template de tarefa, convenção de commit). Encapsula conhecimento, não dados.
- **API direta:** pra integrações pontuais ou quando não existe MCP Server. Mais trabalho, mais controle. **Por que isso importa:** Times que misturam esses conceitos acabam com MCPs pesados ou skills que deveriam ser MCPs. A regra: dado mutável → MCP; conhecimento estável → Skill; integração pontual → API direta.

Pra visualizar: MCP = porta pro mundo externo (mesma imagem dos 4 pilares do Módulo 6). Skill = manual na gaveta. API direta = construir uma porta nova.

Demos guiadas (23 min): o instrutor executa, a plateia acompanha e pergunta.

1. **Configurar um MCP do zero (15 min):** a mesma pergunta sobre versão de biblioteca antes e depois do Context7 — a diferença entre as duas respostas são 4 linhas de configuração, não um modelo melhor.

2. **Playwright (8 min):** o agente abrindo o navegador, logando e verificando uma tela. É a demo mais fácil de entender por quem não é dev.

No kit do pós-seminário: roteiro passo a passo para configurar Context7 + um segundo MCP à escolha, listar as ferramentas descobertas e inspecionar o schema de uma delas.

Módulo 6 — Arquitetura de Contexto: Rules, Skills e Sub-agents (2h)

O que você vai aprender:

- Desenhar a arquitetura de contexto de um projeto com os 4 pilares
- Escrever um Agents.md eficaz (máximo 200 linhas)
- Criar uma skill com **front matter** e **triggers claros**
- Explicar por que **agentes customizados enormes são um antipadrão**

Tópicos:

6.1 A Evolução 2023-2026: De Tudo no Prompt às Skills Modulares

A ideia: A forma como devs interagem com IA passou por 3 fases:

1. **2023-2024 — "Tudo no prompt":** documentação, regras, MCPs, workflows num prompt só. Resultado: contexto gigante, tokens caros, alucinações.
2. **Início de 2025 — Super-agentes:** agentes customizados de 3000+ linhas que faziam tudo. Problema: cada conversa já começava com metade do contexto ocupado.
3. **Meados de 2025 em diante — Skills modulares:** capacidades pequenas, focadas, carregadas sob demanda (sub-agents genéricos). **Por que isso importa:** Se você tá começando agora, pode pular pra fase 3 direto. Não crie agentes enormes. Crie skills pequenas e modulares e use sub-agents genéricos, que identificam as skills necessárias para trabalhar e devolvem o resultado ao agente / processo principal.

Pra visualizar: Fase 1 = todos os ingredientes na panela de uma vez. Fase 2 = robô gigante que gasta metade da energia só pra ligar. Fase 3 = cada ferramenta na sua gaveta/compartimento, utiliza quando precisar.

6.2 Os 4 Pilares da Arquitetura de Contexto

A ideia: Pensa na arquitetura de uma casa:

1. **Rules / Agents.md = Planta baixa:** o QUE é o projeto — estrutura, convenções, arquitetura.
2. **Skills = Cômodos funcionais:** COMO fazer X — capacidades reutilizáveis e independentes.
3. **MCPs = Portas pro mundo externo:** ONDE buscar/alterar dados — Jira, GitHub, Context7, bancos.
4. **Sub-agents = Trabalhadores isolados:** FAÇA isso em paralelo sem poluir meu contexto. **Por que isso importa:** Essa divisão resolve a confusão mais comum: "isso vai como rule, skill ou MCP?"

Pra visualizar: Rules = planta baixa. Skills = ferramentas na caixa. MCPs = portas e janelas. Sub-agents = pedreiros em cômodos diferentes.

6.3 Rules e Agents.md: A Planta Baixa

A ideia: Agents.md (ou CLAUDE.md) é o padrão — um arquivo na raiz do projeto que toda IA lê ao iniciar. Regras:

- **Máximo 200 linhas:** só estrutura de alto nível + ponteiros pra docs detalhadas.
- **Conteúdo típico:** estrutura de diretórios, convenções de código, padrões de commit, links pra docs.
- **Always-apply vs on-demand:** regras críticas sempre carregadas; docs detalhadas sob demanda.
- **Dica pra legados:** peça pra IA analisar o repositório e gerar o Agents.md — Claude Code tem comando `onboard`. **Por que isso importa:** Um Agents.md bem escrito é o que mais multiplica a qualidade das respostas da IA no seu projeto. Sem ele, a IA erra todas as convenções do time.

Pra visualizar: Agents.md = README pra humanos, mas otimizado pra IAs.

6.4 Skills: Capacidades Portáteis

A ideia: Skill é tipo uma classe abstrata — encapsula complexidade e expõe interface simples:

- **Front matter curto:** descrição clara e triggers pra carregamento automático.
- **Portátil:** cria uma vez, usa no Cursor, Claude Code, Copilot. Compartilhável entre times.

- **Exemplo real — Skill Jira Assistant:** encapsula projeto, cloud ID, instância. O dev só diz "corrige CAN-123" e a skill resolve. Sem ela: "corrige CAN-123 na URL techleads.atlassian.net, cloud digital, board CAN..."
- **Trigger automático:** o agente detecta que você mencionou uma task do Jira e carrega a skill. **Por que isso importa:** Skills transformam conhecimento tribal em ativos reutilizáveis. Em vez de cada dev reinventar a roda, o time compartilha skills. Empresas têm criado "marketplaces internos de skills".

Pra visualizar: Skill = classe. MCP = API REST.

6.5 Sub-agents Genéricos: A Revolução Silenciosa

A ideia: A novidade de 2025: você **não** precisa mais criar sub-agents customizados para tudo. Ferramentas modernas (Cursor, Claude Code, Kiro) têm agentes genéricos que:

- Detectam automaticamente quando paralelizar.
- Iniciam processos isolados com contexto limpo.
- Cada sub-agent pode usar skills disponíveis.
- Só o resultado final volta pro contexto principal. **Diferença:**
- **Skill:** NÃO reduz contexto — roda no mesmo processo.
- **Sub-agent:** SIM reduz contexto — processo separado, devolve só o output final. **Por que isso importa:** Antes, sub-agents eram usados pra duas coisas ao mesmo tempo (isolamento + habilidade especializada). Agora cada conceito tem seu lugar: skill = capacidade, sub-agent = isolamento. Você foca em criar boas skills; a ferramenta gerencia os agentes.

Pra visualizar: Skill = manual na sua mesa. Sub-agent = assistente que vai pra outra sala e volta só com a resposta.

6.6 Da Teoria à Prática: Checklist dos 4 Pilares

A ideia:

Pilar	O que colocar	Máximo	Quando carrega
Rules / Agents.md	Estrutura, convenções, ponteiros pra docs	200 linhas	Always ou on-demand

Pilar	O que colocar	Máximo	Quando carrega
Skills	Como criar testes, buscar tasks, gerar release	500 linhas	On-demand (triggers)
MCPs	APIs corporativas: Jira, GitHub, bancos	—	Always (conectados)
Sub-agents	Não crie customizados — ferramentas gerenciam	—	Automático

Por que isso importa: Esse checklist evita o erro clássico: criar um agente de 3000 linhas quando você deveria criar 5 skills de 200 linhas.

Pra visualizar: Canivete suíço que faz tudo mal vs kit de ferramentas especializadas.

Demo guiada (12 min): rodar o comando de onboarding num **legado real e grande** (SIGAA) e ler a saída em voz alta — o que a IA acertou e, sobretudo, o que ela não tinha como saber: a regra de negócio, o motivo histórico de uma decisão, o módulo que ninguém mexe por medo. A conclusão é o ponto do módulo: ela entrega 80% em 3 minutos; os 20% que faltam são o trabalho.

Bloco de perguntas (20 min): reservado no fim do encontro. Online as dúvidas se acumulam no chat em vez de interromperem — sem tempo reservado, nunca são respondidas.

No kit do pós-seminário: gerar o Agents.md do próprio projeto e reduzi-lo a 200 linhas; escrever uma skill com front matter e ajustar a descrição até o trigger disparar sozinho.

Módulo 7 — SDD e Ferramentas na Prática (4h — encontro inteiro)

O que você vai aprender:

- Executar o ciclo SDD completo num cenário real
- Escrever uma spec com user stories, metas e fora de escopo
- Quebrar uma spec em tasks autossuficientes com pré-requisitos
- Comparar ferramentas de IA e selecionar a mais adequada

- Estabelecer um fluxo de documentação viva mantida por IA
- Avaliar a qualidade do código gerado por IA — o que olhar, o que a IA erra, como validar
- Montar um fluxo de validação prático com ferramentas Java **Tópicos:**

7.1 Por que Prompt Direto Falha em Projetos Grandes

A ideia: Quando você joga um PRD gigante num prompt e pede "implementa isso", o resultado é um plano raso que perde contexto. A IA não processa 50 requisitos mantendo coerência — prioriza o início, ignora o meio e alucina no final. É o mesmo problema da janela de contexto cheia, aplicado a desenvolvimento de software.

Por que isso importa: Explica por que times que tentam "codar tudo com IA" sem método falham. SDD não é burocracia — é estratégia pra contornar limitações das LLMs.

Pra visualizar: Prompt direto com PRD gigante = pedir pra um dev implementar 50 features sem dividir em tasks, sem critério de aceite, sem definição de pronto.

7.2 SDD: As 4 Fases

A ideia: Spec-Driven Development em 4 fases:

Fase 1 — Specify: definir problema, metas, user stories, fora de escopo. Alinhamento antes de tudo.

Fase 2 — Design: diagramas, arquitetura, decisões técnicas. Opcional pra projetos pequenos; essencial pra sistemas com múltiplos módulos ou integrações.

Fase 3 — Tasks: quebrar a spec em tarefas autossuficientes. Cada task precisa de: o que fazer, onde (arquivos), pré-requisitos, definition of done. Tasks sem dependências podem rodar em paralelo.

Fase 4 — Execute: sub-agents executando tasks em paralelo, cada um com contexto limpo e isolado — só a task específica mais o contexto relevante.

Por que isso importa: Cada fase tem contexto reduzido e focado. A IA não precisa "lembrar" de tudo.

Pra visualizar: Construção civil. Specify = "quero 3 quartos". Design = planta. Tasks = cronograma. Execute = pedreiros em paralelo.

7.3 STATE.md: Memória Entre Sessões

A ideia: STATE.md persiste o estado do desenvolvimento entre sessões. Guarda decisões e por quê, tasks concluídas, pendentes, blockers e aprendizados. Quando você volta (5 minutos ou 5 dias depois), o agente lê o STATE.md e continua de onde parou.

Por que isso importa: Resolve o maior problema de desenvolver com IA: a perda de contexto entre sessões. Sem STATE.md, cada nova conversa começa do zero.

Pra visualizar: STATE.md = diário de obra. O pedreiro do turno seguinte lê e sabe exatamente o que foi feito.

7.4 Documentação Como Combustível

A ideia: Na era da IA, documentação deixou de ser burocracia e virou combustível. Quanto melhor documentado seu projeto, mais rápido e preciso o agente trabalha. A dica do Waldemar: a cada feature nova, peça pra IA atualizar as guidelines do projeto. Assim a documentação cresce organicamente.

Por que isso importa: Inverte o incentivo: documentar não é mais "perda de tempo que o próximo dev vai aproveitar". É investimento que volta imediatamente na qualidade do código que a IA gera.

Pra visualizar: Documentação = gasolina do motor de IA. Sem gasolina, o motor rateia. Com gasolina de qualidade, roda liso.

7.5 Frameworks SDD: OpenSpec, TLC Spec-Driven e Kiro

A ideia: Em vez de ferramentas de IA genéricas, existem frameworks especializados em SDD que implementam o ciclo Specify → Design → Tasks → Execute. Cada um com sua filosofia e comandos próprios.

OpenSpec — Framework SDD open-source (Fission-AI / MIT)

```
Explore:  /opsx:explore "ideia" → pesquisa viabilidade antes de começar
Propose:  /opsx:propose "feature" → gera proposal.md + specs/ + design.md + tasks.md
Update:   /opsx:update → revisa artefatos e mantém coerência
Execute:  /opsx:apply → implementa tasks do change
```

```
Verify:    /opsx:verify → escaneia código vs artefatos (completeness, correctness, cc
Archive:    /opsx:archive → arquiva change | /opsx:sync → merge delta specs
Install:    npm install -g @fission-ai/openspec && openspec init
```

- Compatível com 25+ ferramentas (Claude Code, Cursor, Copilot, Codex)
- Brownfield-first: projetado para projetos existentes
- Artefatos em `openspec/specs/` e `openspec/changes/`
- `/opsx:verify` retorna issues como CRITICAL, WARNING, SUGGESTION — não gera testes, só valida coerência

TLC Spec-Driven — *Skill adaptativa (Tech Lead's Club)*

```
Setup:      npx skills add ... --skill tlc-spec-driven → .specs/project/ + .specs/code
Specify:     "specify feature [nome]" → .specs/features/x/spec.md (user stories P1/P2/F
Design:      "design feature [nome]" → .specs/features/x/design.md (arquitetura + decis
Execute:     "break into tasks" → .specs/features/x/tasks.md | "implement task [N]" -
Validate:    "validate" / "verify" → valida implementação contra critérios de aceitaçã
Session:     "record decision" / "log blocker" / "pause work" → STATE.md persistido ent
```

- **Auto-sizing:** pula fases automaticamente conforme complexidade (Small → quick, Medium → spec, Large → completo, Complex → pesquisa + plano paralelo)
- Acceptance criteria escritos em formato WHEN/THEN/SHALL durante o Specify
- Gera `.specs/` com análise brownfield: stack, arquitetura, convenções
- Alvo de contexto: <40k tokens com carregamento sob demanda
- Stack-agnostic, funciona no Claude Code, Cursor e Codex
- Gatilhos por linguagem natural (não slash commands)

Kiro — *IDE + CLI com specs integrados (AWS)*

```
Steering:    kiro init (CLI) → structure.md + tech.md + product.md (análise automática
Specify:     /spec new "feature" → requirements.md (EARS) + design.md (Mermaid) + tasks
Execute:     /spec run <name> → implementação autônoma referenciando specs como ground
```

Bugfix: /spec new --bugfix → root-cause analysis com guardrails
 Quick: /spec new --quick → fast-track: faz perguntas antes de gerar o plano

- **Property-Based Testing:** extrai propriedades dos specs e gera varios testes
- **Hooks:** automações event-driven (ex: ao salvar componente, atualiza testes automaticamente)
- Specs viram living documentation que sincroniza com o código
- Suporta feature specs (requirements-first ou design-first), bugfix specs, quick plan

Comparação

Critério	OpenSpec	TLC Spec-Driven	Kiro
Tipo	Framework CLI	Skill adaptativa	IDE + CLI
Open-source	✓ MIT	✓	✗ (freemium)
Auto-sizing	✗	✓	✗
Verificação	/opsx:verify — valida código vs artefatos mas não induz criação de testes unitarios ou e2e	Validate fase — agent verifica contra acceptance criteria (induz a criação e cobertura de testes)	gera testes automaticamente baseados em propriedades (propoe tasks opcionais de testes)
Vendor lock	Nenhum	Nenhum	IDE própria

Por que isso importa: SDD sem framework é possível, mas esses 3 resolvem o problema de "como começar" — cada um com um trade-off diferente entre flexibilidade, estrutura e lock-in.

7.6 RPI: O Ciclo que Alimenta Cada Fase

A ideia: RPI (Research → Plan → Implement) é o micro-ciclo dentro de cada fase do SDD:

- **Research:** abrir o leque — pesquisar abordagens, bibliotecas, patterns.
- **Plan:** salvar o plano em markdown — documentar decisão e racional.
- **Implement:** executar as tasks conforme o plano. **Por que isso importa:** RPI evita o "vibe coding" — codar no feeling sem planejar. Cada decisão tem racional documentado que pode ser revisado depois.

Pra visualizar: RPI = método científico no desenvolvimento: formular hipótese → documentar experimento → executar e observar.

7.7 Avaliando Código Gerado por IA

A ideia: Código gerado por IA que passa nos testes não é sinônimo de código de qualidade. Estudos mostram que código gerado por IA tem alta incidência de code smells, bugs silenciosos e falhas de segurança — mesmo quando os testes passam. Em Java, os problemas mais comuns são vazamento de memória, consultas repetidas ao banco (N+1), senhas/URLs fixas no código e erros engolidos. É papel do desenvolvedor revisar com olhar crítico.

Por que isso importa: Se o time não tem um processo de validação, o débito técnico acumula rápido. O risco vai de código difícil de manter até falhas de segurança em produção. Saber avaliar separa o dev que "usa IA" do dev que "entrega qualidade com IA".

Pra visualizar: Imagine que você pediu pra IA cozinhar um prato. Ela entregou algo bonito e quente — mas você precisa provar pra saber se está bom. Código é igual: "passou no fogo" (testes) não significa que está pronto pra servir.

Ferramentas relacionadas: SonarQube, Checkstyle, PMD, SpotBugs, CodeRabbit

7.8 Na Prática: Como Validar o Código da IA

A ideia: Um fluxo simples de 5 passos ajuda a não pular etapas:

1. **Rode os testes:** `mvn test` ou `gradle test`. Se não passar, nem olhe o resto.
2. **Analizador automático:** SonarQube, Checkstyle. Corrija o que ele apontar.
3. **Peça pra IA revisar:** "Analise esse código como dev sênior — aponte problemas."
4. **Revisão humana (rápida):** A regra de negócio faz sentido? Algo parece estranho?
5. **Teste integrado:** Rode a aplicação completa. Veja se funciona junto. **Por que isso importa:** Esse fluxo é o que times maduros usam. Sem ele, código de

baixa qualidade entra em produção. Com ele, você aproveita a velocidade da IA sem sacrificar qualidade. O segredo é saber quando aplicar todos os 5 passos (código crítico) vs quando simplificar (código simples).

Pra visualizar: É como uma esteira de qualidade numa fábrica. Cada passo filtra um tipo de problema diferente. No final, o que sai é confiável.

Ferramentas relacionadas: CodeRabbit, GitHub Code Review, Claude/ChatGPT/Copilot (como revisor), JUnit, Selenium, Spring Boot Test

Construção ao vivo (1h55): o encontro 4 é uma sessão de construção, não uma aula com laboratório. O instrutor constrói a feature inteira do zero — *matrícula de aprovados em chamada de vagas*, backend Spring Boot + tela Angular — narrando cada decisão, e a plateia acompanha e pergunta.

Bloco 1 — Specify → Design → Tasks (40 min)

Do requisito em português corrido até as tasks revisadas. O momento central é **discordar da spec ao vivo**: a IA decide sozinha o que fica fora de escopo, e isso é uma decisão de produto. Editar aquilo na frente da plateia é a demonstração de que SDD serve para revisar cedo e barato.

Bloco 2 — Execute de verdade (40 min)

As tasks rodam de verdade, com a aplicação subindo ao final e uma matrícula em lote sendo disparada pela tela. O tempo de espera é usado para acompanhar o `tasks.md`, responder o chat e comentar os erros do agente em tempo real. O estágio pré-construído fica apenas como plano B.

Bloco 3 — Validar o código gerado (15 min)

Aplicar o fluxo de 5 passos no código que nasceu 20 minutos antes: testes, analisador estático (com uma correção feita ao vivo) e teste integrado.

Bloco 4 — Quando dá errado: recuperando um agente perdido (20 min)

Quatro falhas provocadas de propósito, com a recuperação de cada uma: contexto poluído (restaurar pelo `STATE.md`), task grande demais (quebrar em três), alucinação de API (apontar a documentação real via MCP) e loop (corrigir a premissa, não o código). É o bloco mais valioso para quem vai usar isso sozinho depois — todo tutorial mostra o caminho feliz.

Bloco de perguntas (15 min): reservado no fim, com abertura para a plateia pedir experimentos ao vivo.

No kit do pós-seminário: templates de `spec.md`, `design.md`, `tasks.md` e `STATE.md`; o repositório com as tags de cada estágio da construção; e requisitos de exemplo para quem não quiser usar um problema do próprio trabalho.

Referências

Vídeos Base (Plano de Nivelamento)

- [Roadmap de IA para DevOps](#) — Fabrício Veronez (20 min)
- [Prompt Engineering — Guia Prático](#) — Fabrício Veronez (2h)
- [SDD: Habilidade #1 para Devs](#) — Waldemar Neto (12 min)
- [Mini Curso IA para Devs](#) — Waldemar Neto (1h30)
- [Rules, Skills, MCPs and Subagents](#) — Waldemar Neto (50 min)

Ferramentas

- `Ollama` — execução local de LLMs open-source
- `Hugging Face` — repositório de modelos, datasets e fine-tunings
- `Cursor` — IDE com multi-agentes, plan mode, skills e MCPs
- `GitHub Copilot` — assistente multi-IDE com CLI
- `Claude Code` — assistente terminal da Anthropic (comando `onboard`)
- `Windsurf` — IDE com browser interno, foco em frontend
- `Cline / Roo Code` — extensões VS Code gratuitas com MCP
- `Continue / Aider` — open-source com foco em planejamento
- `MCP (Model Context Protocol)` — padrão Anthropic pra integração IA ↔ ferramentas
- `RAGAS` — biblioteca pra métricas de qualidade de pipelines RAG

Fontes por Módulo

Módulo 1 — LLMs, Tokens e Fine-Tuning

- [Anthropic Docs: Context Windows](#) — janela de contexto, context rot, boas práticas
- [Anthropic Glossary](#) — definições de token, LLM, RAG, temperatura
- [OpenAI Prompt Guidance](#) — guia oficial de prompting para GPT-5.5
- [OpenAI Prompting](#) — fundamentos de prompt engineering
- [Paper: Attention is All You Need \(Vaswani et al., 2017\)](#) — o paper que criou a arquitetura Transformer
- [Paper: Lottery Ticket Hypothesis \(Frankle & Carbin, 2019\)](#) — redes com 10-20% dos pesos igualam precisão
- [Paper: SparseGPT \(Frantar & Alistarh, 2023\)](#) — remoção de 50-60% dos pesos sem re treino
- [Paper: Wanda \(Sun et al., 2024\)](#) — poda simples por magnitude × ativação

Módulo 2 — Open-Source e Execução Local

- [Ollama Docs](#) — documentação oficial: instalação, CLI, modelos
- [Ollama Model Library](#) — biblioteca de modelos disponíveis
- [Hugging Face Hub Docs](#) — documentação oficial do hub de modelos e datasets

Módulo 3 — Prompt Engineering

- [OpenAI Prompt Engineering Guide](#) — guia oficial com 6 estratégias
- [Anthropic Claude Cookbook: Evaluator-Optimizer](#) — padrão gerador/avaliador
- [OpenAI Prompt Optimization Cookbook](#) — otimização de prompts com GPT-5

Origem dos frameworks de prompt (seção 3.2):

Base acadêmica: - [Language Models are Few-Shot Learners](#) — Brown et al. (OpenAI, 2020). Paper fundacional do GPT-3 que estabeleceu formalmente os conceitos de zero-shot e few-shot prompting. NeurIPS 2020. - [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#) — Wei et al. (Google Brain, 2022). Paper mais citado de prompt engineering; demonstrou que "pensar passo a passo" reduz alucinações em raciocínio. NeurIPS 2022. Ver também: [blog do Google Research](#).

Comunidade de praticantes (sem paper acadêmico): - **RTF (Role-Task-Format)** — sem autor único. Emergiu organicamente na comunidade de usuários do ChatGPT após o lançamento do GPT-3.5 (2022-2023). Referência de entrada: [Mastering ChatGPT Prompting: The RTF Framework](#) (Medium, ago/2023). Mencionado academicamente em contexto clínico: [PubMed/NIH \(2025\)](#). - **CO-STAR (Context-Objective-Style-Tone-Audience-Response)** — origem rastreável: criado pela

equipe de Data Science & AI do GovTech Singapore (2023). Sheila Teo ganhou a competição nacional de prompt engineering de Singapura com a abordagem. Post original: [How I Won Singapore's GPT-4 Prompt Engineering Competition](#). Formalizado no [Prompt Engineering Playbook oficial do GovTech](#). - **CARE (Context-Action-Result-Example)** — sem autor único; variações da sigla existem. Referências: [Nielsen Norman Group](#) (UX), [NIH/PubMed](#) (saúde). - **RISE (Role-Input-Steps-Expectation)** — sem autor único. Origem difusa na comunidade de praticantes (2022–2023). Evoluiu para RISEN com "Narrowing" adicionado. Comparativo entre frameworks: [Prompt Engineering Frameworks — RACE, CARE, APE, CREATE, TAG, RISE](#) (Medium, 2025).

Módulo 4 — RAG, Embeddings e Graph RAG

- [LangChain Retrieval Guide](#) — pipeline RAG completo: loading, splitting, embedding
- [LangChain: Build a RAG Agent](#) — tutorial passo a passo com Claude
- [Paper: GraphRAG \(Microsoft, 2024\)](#) — RAG baseado em grafos de conhecimento
- [Paper: Lost in the Middle \(Liu et al., 2023\)](#) — degradação com contexto cheio

Módulo 5 — MCP e Agentes

- [MCP Official Specification](#) — especificação completa do protocolo
- [MCP Documentation](#) — guias de implementação
- [MCP GitHub \(TypeScript SDK\)](#) — SDK oficial
- [MCP GitHub \(Python SDK\)](#) — SDK oficial
- [Anthropic Blog: Common Workflow Patterns](#) — patterns de agentes: quando usar cada um

Módulo 6 — Arquitetura de Contexto

- [Anthropic: Code Review \(Managed Service\)](#) — code review multi-agente oficial
- [Claude Blog: Code Review](#) — como a Anthropic usa code review internamente

Módulo 7 — SDD e Avaliação de Código

- [OpenSpec GitHub](#) — framework SDD open-source (MIT, 48k+ stars)
- [OpenSpec Docs](#) — documentação oficial

- [Paper: Assessing AI-Generated Code Quality \(Sonar, 2025\)](#) — 4.442 tarefas Java, 5 LLMs — nenhuma correlação entre passar teste e qualidade



Glossário do Workshop

Módulo 1 — Fundamentos

- **LLM** (Large Language Model): Modelo de IA treinado com bilhões de textos para prever e gerar linguagem. Ex: GPT, Claude, Gemini. *Módulo 1*
- **Token**: Unidade de processamento da IA — um pedaço de texto (palavra, sílaba ou caractere). "Inteligência artificial" = 2 tokens. *Módulo 1*
- **Tokenização**: Processo de quebrar texto em tokens. Diferente de contar palavras. *Módulo 1*
- **Janela de contexto** (Context Window): Quantidade máxima de tokens que o modelo consegue "ver" de uma vez. É a RAM da IA. *Módulo 1*
- **Autocomplete**: Mecanismo de prever a próxima palavra. LLMs são autocompleters turbinados. *Módulo 1*
- **Alucinação**: Quando a IA inventa uma resposta com confiança, mas ela é falsa. *Módulo 1*
- **Data de corte**: Data limite do conhecimento do modelo. O modelo não sabe o que aconteceu depois. *Módulo 1*
- **Não-determinismo**: A mesma entrada pode gerar respostas diferentes. Inerente a modelos probabilísticos. *Módulo 1*
- **Parâmetros**: Números internos que o modelo ajusta durante o treinamento (pesos das conexões). *Módulo 1*
- **Temperatura**: Controla o nível de criatividade. 0 = sempre a mesma resposta (determinístico). 1+ = mais variação. *Módulo 1*
- **Top-p**: Alternativa à temperatura. Controla o vocabulário considerado: pega os tokens mais prováveis até somar p% de probabilidade. *Módulo 1*
- **Pré-treinamento**: Fase onde o modelo é treinado com bilhões de textos para aprender padrões da linguagem. Custa milhões de dólares. *Módulo 1*
- **Fine-tuning**: Ajuste fino do modelo pré-treinado para uma tarefa ou domínio específico. *Módulo 1*
- **Multimodal**: Capacidade de processar mais de um formato (texto, imagem, áudio, vídeo) simultaneamente. *Módulo 1*

- **Kilowatt da IA:** Analogia: token é a unidade de consumo, como kilowatt na conta de luz. *Módulo 1*
- **Prompt Caching:** Técnica onde APIs guardam instruções repetidas e cobram ~90% menos na reutilização. Tipo git: clone vs pull. *Módulo 1*

Módulo 2 — Modelos Locais vs Cloud

- **SLM** (Small Language Model): Modelo pequeno que roda localmente (ex: Phi-3, Llama 8B). Entrega 80% da qualidade por 10% do custo. *Módulo 2*
- **Ollama:** Ferramenta para rodar modelos locais (Llama, Phi-3) sem depender de internet ou API key. *Módulo 2*
- **vLLM:** Engine de alta performance para servir modelos LLM. Suporta batching dinâmico, PagedAttention. *Módulo 2*
- **llama.cpp:** Biblioteca em C++ para executar modelos quantizados em CPU/GPU de consumidor. *Módulo 2*
- **TPS** (Tokens Por Segundo): Métrica de velocidade de geração — quantos tokens o modelo produz por segundo. *Módulo 2*

Módulo 3 — Prompt Engineering

- **Prompt:** A instrução ou pergunta que você envia para a IA. É o "input" do modelo. *Módulo 3*
- **Prompt Engineering:** Arte de construir prompts eficazes para obter respostas de qualidade. *Módulo 3*
- **Role** (Papel): Definir quem a IA deve "ser" no prompt (ex: "Você é um arquiteto Java sênior"). *Módulo 3*
- **System Prompt:** Instrução fixa que define o comportamento do agente, enviada antes de cada conversa. *Módulo 3*
- **Zero-shot:** Pedir algo à IA sem dar exemplos. Funciona bem para tarefas simples. *Módulo 3*
- **Few-shot:** Dar 2-3 exemplos no prompt antes de pedir a tarefa. A técnica mais subestimada. *Módulo 3*
- **Chain of Thought** (CoT): Pedir para a IA "pensar passo a passo" antes de responder. Reduz alucinações. *Módulo 3*
- **RTF** (Role-Task-Format): Framework de prompt em 3 etapas: defina o papel, a tarefa e o formato da resposta. *Módulo 3*
- **CARE** (Context-Action-Result-Example): Framework com contexto, ação, resultado esperado e exemplo. *Módulo 3*

- **RISE** (Role-Input-Steps-Expectation): Framework para tarefas complexas: papel, entrada, passos, expectativa. *Módulo 3*

Módulo 4 — RAG e Embeddings

- **RAG** (Retrieval-Augmented Generation): Técnica que busca documentos relevantes e injeta no contexto da IA, sem precisar treinar o modelo. *Módulo 4*
- **Embedding**: Representação numérica do significado de um texto (vetor). É o "GPS no espaço do significado". *Módulo 4*
- **Chunking**: Quebrar documentos grandes em pedaços menores (ex: 500 tokens) para busca eficiente. *Módulo 4*
- **Semantic Search**: Busca por significado (usando embeddings), não por palavras exatas. *Módulo 4*
- **Hybrid Search**: Combina busca semântica + busca por palavras-chave (BM25) para melhores resultados. *Módulo 4*
- **BM25**: Algoritmo clássico de busca por palavras-chave. Versão moderna do TF-IDF. *Módulo 4*
- **Re-ranking**: Pegar os resultados da busca e reordenar por relevância real. Normalmente usa cross-encoder. *Módulo 4*
- **Cross-encoder**: Modelo que compara dois textos lado a lado e dá uma nota de similaridade. Mais preciso, mais lento. *Módulo 4*
- **Bi-encoder**: Modelo que converte cada texto em embedding separadamente. Mais rápido para busca em larga escala. *Módulo 4*
- **PCA** (Principal Component Analysis): Técnica que reduz N dimensões para as mais importantes. "A máquina que ranqueia o que importa." *Módulo 4*
- **Graph RAG**: RAG que extrai entidades e relações de documentos em um grafo, depois consulta o grafo. Custo 5-10x maior. *Módulo 4*
- **Cypher**: Linguagem de consulta para bancos de grafos (Neo4j). Usada em Graph RAG. *Módulo 4*
- **Neo4j**: Banco de dados orientado a grafos. Às vezes usado como armazenamento para Graph RAG. *Módulo 4*

Módulo 5 — MCP, Agentes e Ferramentas

- **MCP** (Model Context Protocol): Protocolo universal para conectar IAs a ferramentas externas (APIs, bancos, arquivos). Padrão mais adotado em 2026. *Módulo 5*

- **Ferramenta** (Tool): Uma função que a IA pode chamar (ex: buscar no Jira, criar repositório). *Módulo 5*
- **Function Calling**: Mecanismo onde o modelo retorna uma chamada de função estruturada, executada pelo seu código. *Módulo 5*
- **Agente**: Looping autônomo onde a IA recebe uma tarefa, escolhe ferramentas, executa, observa, até completar. *Módulo 5*
- **Sub-agente**: Agente especializado criado por outro agente para delegar sub-tarefas. *Módulo 5*
- **Skill**: Conhecimento procedural estável encapsulado (ex: "como criar testes unitários"). É o "README da IA". *Módulo 5*
- **Agents.md**: Arquivo de configuração de agentes no Claude Code. Define ferramentas, instruções e comportamentos. *Módulo 5*
- **CLAUDE.md**: Arquivo de instruções globais do projeto para o Claude Code. Guia de estilo, convenções, regras. *Módulo 5*
- **SDD** (Specification-Driven Development): Metodologia que usa especificações formais (spec.md) para guiar a IA. *Módulo 5*
- **STATE.md**: Arquivo de estado que preserva o progresso entre sessões da IA. Evita retrabalho. *Módulo 5*
- **Prompt Injection**: Ataque onde instruções maliciosas na entrada do usuário sequestram o comportamento da IA. *Módulo 5*
- **Excessive Agency**: Quando a IA tem mais permissões do que deveria, podendo executar ações destrutivas. *Módulo 5*
- **Tool Poisoning**: Ataque onde uma ferramenta da IA é comprometida para retornar dados falsos. *Módulo 5*
- **LangGraph**: Framework para construir agentes como grafos de estados (ciclos, forks, joins). *Módulo 5*
- **CrewAI**: Framework para orquestrar múltiplos agentes colaborando em papéis definidos. *Módulo 5*
- **AutoGen**: Framework da Microsoft para agentes conversacionais multi-agente. *Módulo 5*

Módulo 6 — Arquitetura de Contexto

- **Arquitetura de Contexto**: Metodologia para organizar instruções da IA em camadas (Agents.md, Skills, MCPs, SDD) para contexto limpo e barato. *Módulo 6*
- **Estado** (State): Informação preservada entre interações com a IA. O STATE.md é o gerenciador de estado. *Módulo 6*

Módulo 7 — SDD e Ferramentas

- **Transformer:** Arquitetura de rede neural que revolucionou a IA. Base de todos os LLMs modernos (GPT, Claude, Gemini). *Módulo 7*
- **Self-attention:** Mecanismo do Transformer que calcula a importância de cada palavra em relação às outras. *Módulo 7*
- **RAGAS** (RAG Assessment): Framework para medir qualidade de RAG — fidelidade, relevância, precisão do contexto. *Módulo 7*
- **LLM-as-Judge:** Usar um LLM para avaliar a qualidade da saída de outro LLM. *Módulo 7*

Transversal

- **OpenCode:** Ferramenta de AI coding integrada ao terminal do VS Code. Usada como ferramenta principal nos laboratórios. *Transversal*
- **Cursor:** IDE com AI integrada. Alternativa ao OpenCode. *Transversal*
- **Claude Code:** CLI da Anthropic para coding assistido por IA. Alternativa ao OpenCode. *Transversal*
- **Copilot** (GitHub Copilot): AI pair programmer da Microsoft. Alternativa ao OpenCode. *Transversal*
- **OpenAPI:** Especificação para descrever APIs REST. Usada por MCP Servers. *Transversal*
- **LSP** (Language Server Protocol): Protocolo para autocomplete e diagnósticos em IDEs. *Transversal*
- **CI/CD:** Pipeline de integração e deploy contínuos. Automatiza build, teste e publicação. *Transversal*
- **GIGO** (Garbage In, Garbage Out): Se você dá entrada de baixa qualidade para a IA, a saída será igualmente ruim. *Transversal*
- **Algoritmo Determinístico:** Função que sempre retorna o mesmo resultado para a mesma entrada. Ao contrário de LLMs. *Transversal*